

AiSD

WYKZAD II

Algorytmy zadaniowe

Przykład.

Dane: $c_1, \dots, c_k \in \mathbb{N} \rightarrow$ nominały monet
 $R \in \mathbb{N} \rightarrow$ kwota do wydania

Zadanie:

Predstawienie R w postaci:

$$R = \sum_{i=1}^k \alpha_i c_i \quad \alpha_i \in \mathbb{N}$$

(Czyli wydajemy R za pomocą nominałów)
tak aby ilość tych monet było najmniejsza
zn. $\sum_{i=1}^k \alpha_i$ było minimum
np.

$$R = 77$$

$$cl := [1, 2, 5, 10, 20, 50]$$

Propozycja algorytmu:

Bierzemy kolejne nominały od największego do najmniejszego bierzemy ich tyle ile mieści się

np.

$$(1) R = 77$$

$$cl := [1, 2, 5, 10, 20, 50]$$

$$77$$

$$= 50 + 20 + 5 + 2$$

$$(2) R = 13$$

$$cl := [1, 5, 8, 9]$$

Algorytm podał:

$$13 = 9 + 4 \cdot 1$$

$\rightarrow 5$ nominałów

Tymczasem optimum to:

$$13 = 8 + 5$$

$\rightarrow 2$ nominały :-)



Zestawy nominalów dla których powyższy algorytm zwróci optymalne rozwiązanie:

1° $c[] = [1, 100]$

dowód przez stwierdzenie czytelnikowi, jako ćwiczenie, gdyż jest prosty :)

Moja idea jest taka aby posortować opta i rozwiązanie algorytmu malejąco, wziąć 1st nominal gdzie się różni i pokazać, że jego istnienie prowadzi do sprzeczności (tj. opt wydał złe resztki albo $\exists$ coś optymalniejszego, niż OPT)

2° $c[] = [1, 2, 5, 10, 20, 50, \dots, 10^k, 2 \cdot 10^k, 5 \cdot 10^k, 10^{k+1}, \dots]$

d-d:

do uzupełnienia, prawdopodobnie tak jak wyżej.

3° $c[] = [F_0, F_1, F_2, \dots]$ gdzie F_i - to i-ta liczba fibonacciego

d-d:

to jest zad. z ćwiczeń, poprzedzające i wkleję tu.

Problem:

Dane $c_1, c_2, \dots, c_k$

Pytanie: Czy dla tych nominatów powyższy zachłanny algorytm daje optymalne rozwiązanie?

NP - trudny??

PROBLEM (MST)

Dane: graf $G(V, E, c)$ $c: E \rightarrow \mathbb{N}$

Zadanie: znaleźć $E', E' \subseteq E$

• E' jest drzewem

• $\sum_{e \in E'} c(e)$ jest minimalne

Strategie zachłanne:

1° algorytm Kruskala: ustawiamy krawędzie

w porządku $e_1 \leq e_2 \leq \dots \leq e_m$

• $E' := \emptyset$
• Wstawiamy kolejne krawędzie wg reguły
jeżeli e_i wstawione do drzewa nie
tworzy cyklu wstawiamy do E'

Pytanie Jak sprawdzić, czy w drzewie jest cykl?

2° algorytm Primera:

$E' := \emptyset$

$V' := \emptyset$

$v \rightarrow$ dowolny wierzchołek

$V' \cup \{v\}$

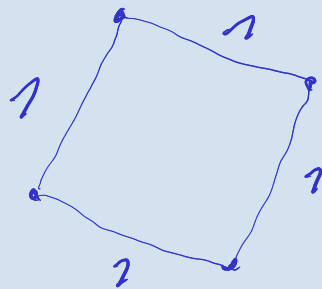
Dopóki $V' \neq V$:

Wybierz krawędź
najtańszą i niepowodującą
do E' i dołącz
do E' i dołącz
tej krawędzi do V'

3^o algorytm Boruvki (Sollina)

$$G_0 := G(V, \emptyset)$$

Dopóki w G_0 istnieje spojne składowe:
połącza je najtańsze krawędzie między nimi



$\forall$ krawędzi $\langle v_i, v_j \rangle$
podtrzymujemy leksykograficznie:

$$\langle c(e), i, j \rangle$$

$$\langle c(e'), i', j' \rangle$$

Minimalność drzewa Kruskala:

Niech $f_1, \dots, f_{n-1}$ - krawędzie wybierane przez
algorytm Kruskala. Załóżmy nie uprzed, że
istnieje rozwiązanie lepsze, oznaczmy krawędzie
lepszego drzewa przez $e_1, e_2, \dots, e_{n-1}$

$$OPT = \sum_{i=1} c(e_i) < \sum_{i=1} c(f_i)$$

Weźmy takie e_i i f_i - $\forall k, j \leq i$ $c(e_j) = c(f_j)$

oraz $c(e_i) \neq c(f_i)$. Rozważmy przypadki:

$$c(e_i) < c(f_i):$$

Wtedy alg Kruskala rozciągnąłby e_i wcześniej,
ale skoro jej nie wybrał, tworzyłaby cykl
z pozostałymi $e_j \leq i$ ale założyłismy, że
 $f_1, \dots, f_{i-1} = e_1, \dots, e_{i-1}$, więc OPT ma cykl

$c(o_i) > c(f_i)$ do przeczytania w notatce

TOOO - fajnie uzupełnić i mieć wszystkie zachłony
tutej

Szeregowanie zadań - (czy też „Jak się wpada na zachłana?”)

Idea - procesor 1-włtkowy wykonuje zadania jeden po drugim - tj. nie zacznie i-tego póki i-1 będzie wykonywane
Dane:

p_i - czasy wykonania zadań

Cel:

Ustalić kolejność wykonywania π zadań, aby minimalizować

T_i = czas przebywania i-tego zadania w systemie, tj:

$$\begin{cases} T_1 = p_1 \\ T_i = T_{i-1} + p_i \end{cases}$$

Jak wpasnąć na zachłana?

metoda by ATA, i pewnie nie tylko.

Weźmy dowolną kolejność π .

Rozważmy π' takie że π' powstanie przez swap 2 sąsiednich elementów π_k oraz π_{k+1} . T_i - czas przebywania i-tego zadania w perm. π , a $T'_{\pi'_i}$ w perm. π'

$$\pi: \sum_{i=1}^n T_i = \sum_{i=1}^{k-1} T_i + T_k + T_{k+1} + \sum_{i=k+2}^n T_{\pi_i}$$

$$\pi': \sum_{i=1}^n T'_i = \sum_{i=1}^n T'_i + T'_k + T'_{k+1} + \sum_{i=k+2}^n T'_i$$

Pytanie: kiedy $\sum_{i=1}^n T_i < \sum_{i=1}^n T'_i$?

$$\sum_{i=1}^{k-1} T_i + T_k + T_{k+1} + \sum_{i=k+2}^n T_{\pi_i} < \sum_{i=1}^n T'_i + T'_k + T'_{k+1} + \sum_{i=k+2}^n T'_i$$
$$\underbrace{T_{k-1} + p_{\pi_k} + T_{k-1} + p_k + p_{\pi_{k+1}}}_{p_{\pi_k}} < \underbrace{T_{k-1} + p_{\pi_{k+1}} + T_{k-1} + p_{\pi_{k+2}} + p_k}_{p_{\pi_{k+1}}}$$
$$p_{\pi_k} < p_{\pi_{k+1}}$$

Bo zamieniliśmy tylko kolejność 2 sąsiednich elementów. Wtedy

$$\begin{aligned} T'_{k+2} &= T'_{k-1} + p_{\pi_{k+1}} + p_{\pi_k} + p_{\pi_{k+2}} \\ &= T_{k-1} + p_{\pi_k} + p_{\pi_{k+1}} + p_{\pi_{k+2}} = T_{k+2} \end{aligned}$$

Reszta indukcyjnie.

Czyli chyba wystarczy posortować je reszeczko po czasach trwania.

Niby oczywiste - ale czasami to błąd dzięki proporcje, a nie pojedyncze parametry i je już trudno zgodzić :-/.

Tw. Ta strategia działa...

d-d:

p - rozwiązanie optymalne.

Jeśli $\exists x < y$, że $p_x > p_y$, znajdź takie $x \leq i < j$, że $p_i > p_{i+1}$ (inaczej porządek od p_x ciąg p_i byłby cały czas rosnący i $p_x < p_y$). Swapujemy p_i oraz p_{i+1} tworząc p' . Z powyższej obserwacji wynika, że p' jest optymalniejsze od p , a to sprzeczne z jego optymalnością.

Powyższy dowód można uogólnić, wtedy wystarczy pokazać optymalność dla pary wybieranych elementów.

Set-Cover

Dane: U - uniwersum

$$S = \{S_1, S_2, \dots, S_k\} \quad S_i \subseteq U$$

$$c: S \rightarrow \mathbb{R}_+$$

Zadanie:

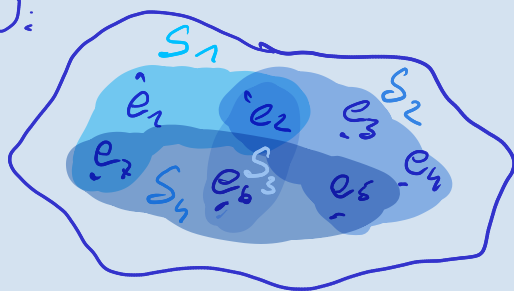
znaleźć najtańszą podrodzinę S (oznaczymy przez C)
pokrywającą U .

$$\bigcup_{X \in C} X = U$$

$$\text{Koszt}(C) = \sum_{X \in C} c(X)$$

Przykład:

U :



$$\begin{aligned} S_1 &= \{e_1, e_2, e_3\} & c(S_1) &= 8 \\ S_2 &= \{e_2, e_3, e_4, e_5\} & c(S_2) &= 12 \\ S_3 &= \{e_2, e_6\} & c(S_3) &= 6 \\ S_4 &= \{e_5, e_6, e_7\} & c(S_4) &= 9 \end{aligned}$$

$$C' = \{S_1, S_2, S_3\} \quad \text{Koszt}(C') = 8 + 12 + 6 = 26$$

$$C'' = \{S_1, S_2, S_4\} \quad \text{Koszt}(C'') = 8 + 12 + 9 = 29$$

Strategie zachłanne (Kiepskie):

- ~ najtańszy podzbiór
- ~ najliczniejszy podzbiór
- ~ podzbiór zawierający największą liczbę niepokrytych elementów U

Ten problem jest NP-trudny $\Rightarrow$ wielomianowe
(a w szczególności zachłanne) rozwiązania nie istnieją.

Algorytm aproksymacyjny:

C - dotychczas wybrane zbiory

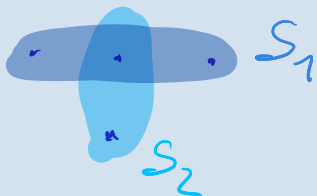
$$cne(S_i) = \frac{c(S_i)}{|S_i \setminus \bigcup_{S \in C} S|}$$

$$c(S_i)$$

określamy to przez $niep(S_i)$

cne to intuicyjnie "cena per element"

te elementy z S_i , które nie są jeszcze pokryte



$$c(S_1) = 10$$

$$c(S_2) = 8$$

$$cne(S_1) = \frac{8}{3}, \text{ gdy } C = \emptyset$$

$$cne(S_2) = \frac{8}{2} = 4, \text{ gdy } C = \emptyset$$

$$cne(S_1) = \frac{10}{2} = 5, \text{ gdy } C = \{S_2\}$$

$$cne(S_2) = \frac{8}{1} = 8, \text{ gdy } C = \{S_1\}$$

Wybieramy rozwiązanie S_i o najmniejszej cenie



$$C := \emptyset$$

it 1 ceny:

$$c(S_3) = 12$$

$$c(S_1) = 3$$

$$c(S_2) = 30$$

$$cne(S_1) = \frac{3}{3} = 1$$

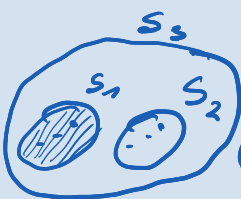
$$cne(S_2) = \frac{30}{3} = 10$$

$$cne(S_3) = \frac{12}{6} = 2$$

3 elementy z S_1 nie należą do C

— || — S_2 — || —

6 elementów z S_3 nie należą do S_1



$$C := C \cup S_1$$

it 2

$$cne(S_2) = \frac{30}{3} = 10$$

$$cne(S_3) = \frac{12}{3} = 4$$

$$C := C \cup S_3$$

3 elementy z S_2 nie należą do C

3 elementy z S_3 nie należą do C



Wszystkie el. pokryte $\Rightarrow$ kończymy algorytm.

$$C = \{S_1, S_3\}, \text{ a opt zawiera tylko } S_3.$$

Przyjmujemy, że zbiór S_2 pokryty elementami ma $cne(S) = \infty$, do nie opłaca się go brać.

Pw. ta strategia daje rozwiązanie o koszcie
 $OPT \cdot \log n$ gdzie n to liczba elementów.
Lemat:

Niech $e_1, \dots, e_n$ są elementami pokrywanymi
 kolejno przez algorytm oraz $c(e_i) = c(S)$,
 t.j. e_i jest pokrywany przez S . Wówczas:

$$c(e_i) \leq \frac{OPT}{n-i+1}$$

d-d:

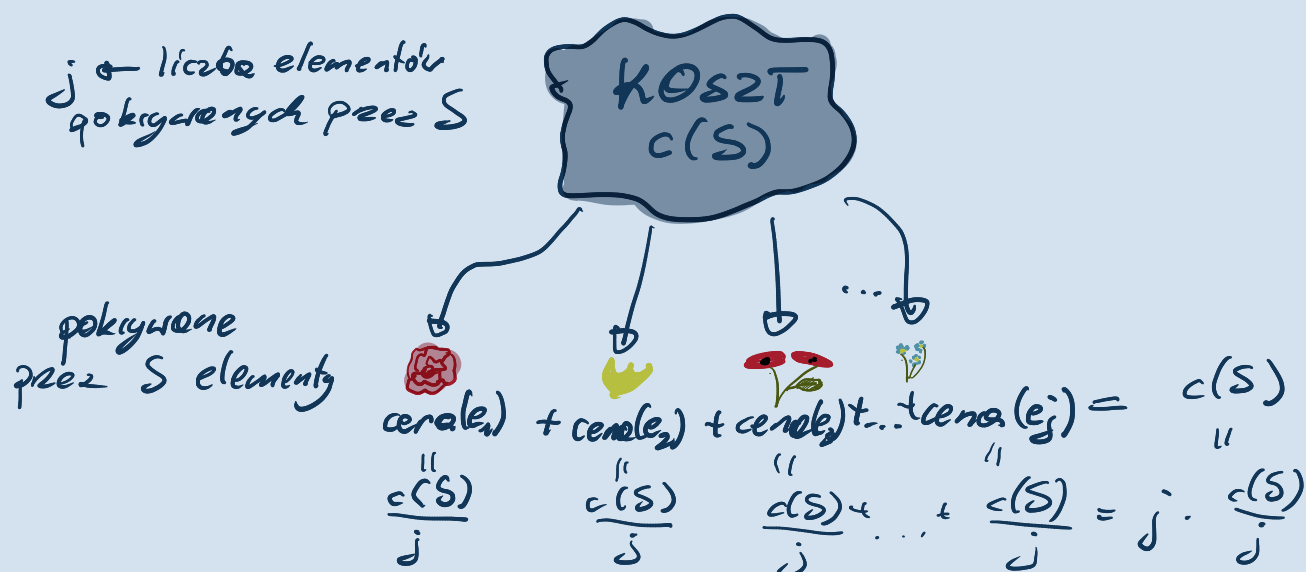
Zauważmy, że $e_1 \dots e_{i-1}$ są pokryte.

Pokrywamy e_i .

Zastanówmy się, co by było, gdybyśmy te $n-i+1$
 pozostałych elementów pokryli zbiorami z opty?

W najgorszym przypadku uzyskujemy $\emptyset$ zbioru
 z OPT , co da koszt OPT .

Zauważmy, że ten koszt zostanie rozdstrybuowany na
 elementy (w postaci cen), bo koszt każdego zbioru rozdzielimy
 na pokrywane elementy



Średnia cena elementów w tym pokryciu wynosi

$\frac{OPT}{n-i+1}$, związku z tym istnieje element o cenie nie większej niż ta średnia (przy pokryciu z zbiorami z OPT_0). Niech e_i będzie zbiorem zawierającym ten element. Na wszelki wypadek upewnijmy się, że S_{OPT} pokrywa wszystko z $\{e_i \dots e_n\} \cap S_{OPT}$, wtedy cna S_{OPT} spadnie, a ewentualnych sąsiadów (też tworzącego elementy z $\{e_i \dots e_n\} \cap S_{OPT}$) wzrośnie. Takiego S_{OPT} algorytm rozwiązy przy przypisaniu cen, ale bierze elementy o rosnącym cenie, czyli

$$cna(e_i) \leq cna(S_{OPT}) \leq \frac{OPT}{n-i+1}$$

Co daje też.

Zatem koszt algorytmu:

$$\sum_{\text{zbiory}} S_{\text{zbiory}} = \sum_{i=1}^n cna(e_i) \leq OPT \sum_{i=1}^n \frac{OPT}{n-i+1} \propto OPT \cdot \log n$$

bo koszt każdego zbioru dystrybuujemy na elementy

Tw Niech $\forall S_i \quad c(S_i) = 1$, a d to
 moc największego zbioru, wówczas rozwiązanie
 algorytmu nie koszt $\leq OPT \sum_{i=1}^d \frac{1}{i}$, czyli $OPT \cdot \log d$
d-d:

Oznaczenia jak w wyżejszym dowodzie.

Niech T będzie dowolnym zbiorem z opt i
 niech ma moc k . Niech $t_1, \dots, t_k \rightarrow \text{elementy} \in T$
 w kolejności pokrycia przez algorytm.

Złożymy, że $t_1, \dots, t_{i-1}$ są pokryte. Rozważmy
 moment pokrywania t_i

$$\text{cena}(t_i) \leq \text{cna}(T) = \frac{c(T)}{\underbrace{k-i+1}_{\substack{\text{tyle elementów} \\ \text{z } T \text{ jest jeszcze} \\ \text{niepokrytych}}}} = \frac{1}{k-i+1}$$

Wówczas

$$\sum_{i=1}^k \text{cena}(t_i) \leq \sum_{i=1}^k \frac{1}{k-i+1} = \sum_{i=1}^k \frac{1}{i}$$

Byli $\forall T \in opt$ suma cen elementów jest $\leq \sum_{i=1}^{|T|-1} \frac{1}{i}$

Zatem

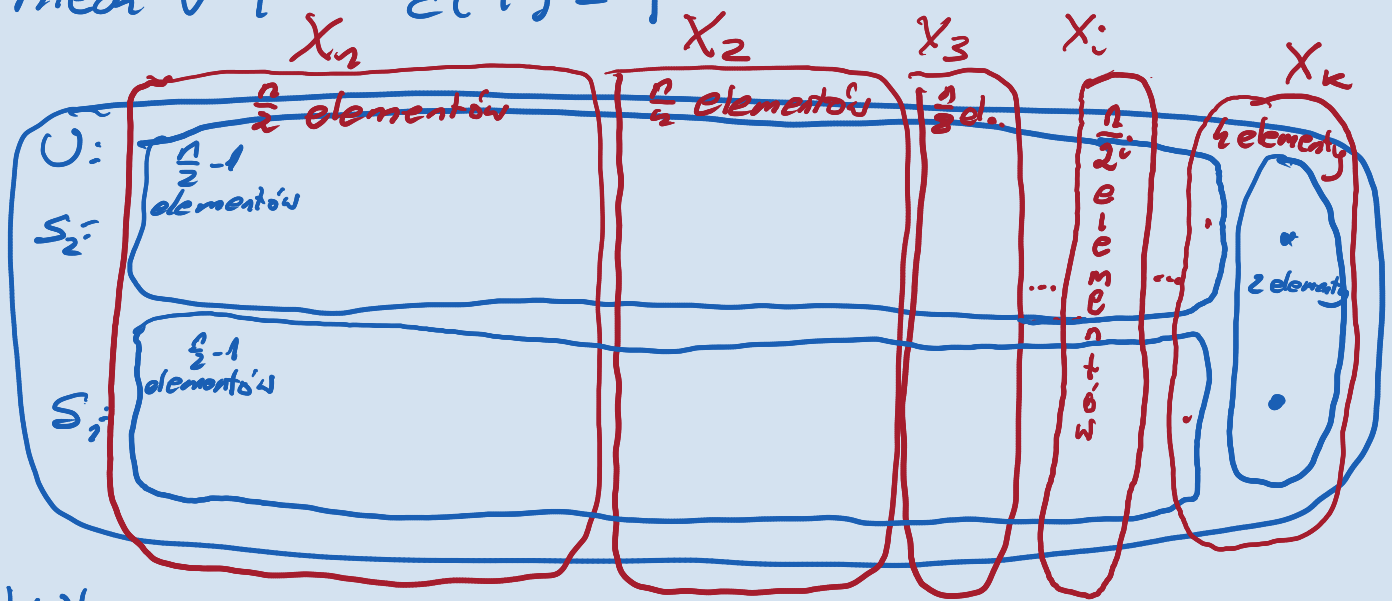
$$\sum_{S \in S^{ALG}} \text{cna}(S^{ALG}) = \sum_{i=1}^n \text{cena}(e_i) \leq \sum_{\forall T \in opt} \underbrace{\sum_{e_i \in T} \text{cena}(e_i)}_{\downarrow} \leq$$

każdy element może zostać pokryty przez zbiór z OPT , dlatego
 każdy zostanie uwzględniony w tej sumie przynajmniej raz

$$\leq \sum_{\forall T \in opt} \left(\sum_{i=1}^{|T|-1} \frac{1}{i} \right) = \left\{ \begin{array}{l} |T|-1 \leq d-1 \\ \text{bo } d \text{ to największa} \\ \text{moc} \end{array} \right\} = \sum_{\forall T \in opt} \left(\sum_{i=1}^{d-1} \frac{1}{i} \right) =$$

$$= \left(\sum_{i=1}^d \frac{1}{i} \right) \cdot \left(\sum_{\forall T \in opt} 1 \right) = \sum_{i=1}^d \frac{1}{i} \cdot \left(\sum_{\forall T \in opt} c(T) \right) = \sum_{i=1}^d \frac{1}{i} \cdot OPT$$

Czy analizę da się poprawić? — **NIE**
 Przykład
 Niech $\forall T \quad c(T) = 1$



$$|U| = n$$

S_1, S_2 zawierają $\frac{n-2}{2} = \frac{n}{2} - 1$ elementów S_3 - zawiera 2 elementy
 S_1, S_2, S_3 - rozłączne

$X_1, \dots, X_k$, t.j. X_i zawiera $\frac{n}{2^i}$ elementów

Oraz $\forall X_i \subseteq X_j \Rightarrow X_i \subseteq S_1 \cup S_2 \wedge X_i \not\subseteq S_3$. $X_k \subseteq S_1 \cup S_2 \wedge X_k \not\subseteq S_3$
 $\forall 1 \leq i \leq k-1 \quad X_i = X_i^{(1)} \cup X_i^{(2)} \wedge |X_i^{(1)}| = |X_i^{(2)}| \wedge X_i^{(1)} \cap X_i^{(2)} = \emptyset \wedge X_i^{(1)} \subseteq S_1 \wedge X_i^{(2)} \subseteq S_2$
 Po czego nie wypisałem - wypisać proszę z rysunku

TW: Po i -tym kroku:
 $\rightarrow$ algorytm wybiera X_i
 $\rightarrow S_1, S_2$ mają $\frac{n}{2} - 1 - \sum_{j=1}^i \frac{n}{2^j}$ elementów

d-d:
 W 1 kroku $cne(S_2) = cne(S_1) = \frac{1}{\frac{n-2}{2}} = \frac{2}{n-2}$
 ale $cne(X_1) = \frac{1}{\frac{n}{2}} = \frac{2}{n} < \frac{2}{n-2}$,
 zatem algorytm weźmie X_1

W i -tym ($1 < i < k$)
 $C := \{X_j : 1 \leq j \leq i-1\}$

S_1 i S_2 mają po $\frac{n}{2} - 1 - \sum_{j=1}^{i-1} \frac{n}{2^j}$ elementów

$$n \cdot \sum_{j=1}^{i-1} \frac{1}{2^{j+1}} = n \cdot \sum_{j=2}^i \frac{1}{2^j} = n \cdot \frac{1}{4} \cdot \sum_{j=2}^i \frac{1}{2^{j-2}} = \frac{1}{4} n \cdot \sum_{j=0}^{i-2} \frac{1}{2^j} = \frac{1}{4} n \cdot \frac{1(1 - (\frac{1}{2})^{i-1})}{\frac{1}{2}} = \frac{1}{2} n (1 - \frac{1}{2^{i-1}})$$

Różnica to daje $\frac{n}{2} - 1 - \frac{1}{2} n (1 - \frac{1}{2^{i-1}}) = \frac{n}{2} - 1 - \frac{n}{2} + \frac{n}{2^i} = \frac{n}{2^i} - 1$

Wtedy X_i ma $\frac{n}{2^i}$ elementów, to znacząco więcej niż $\frac{n}{2^i} - 1$

Łatwy algorytm wybiera X_i bo ma więcej elementów
niepokrytych

Liczba niepokrytych elementów S_1, S_2 spadnie
po równo $\frac{1}{2^i} \cdot \frac{1}{2} = \frac{1}{2^{i+1}}$, bo X_i pokryje wszystkie swoje
elementy, a połowę z nich jest w S_1 , a połowę w S_2 .

Czyli liczba niepokrytych elementów S_1, S_2 wyniesie
w następnej iteracji $\frac{n}{2} - 1 - \sum_{j=1}^{i-1} \frac{1}{2^{j+1}} - \frac{1}{2^{i+1}} = \frac{n}{2} - 1 - \sum_{j=1}^i \frac{1}{2^{j+1}}$

$i=k$:

S_1, S_2 mają tylko 1 element niepokryty

S_3 - 2 elementy

X_k - 4 elementy $\rightarrow$ ten zostanie wybrany

Zadanie dla czytelnika - uprościć dowód.

Oto wniosek:

Algorytm wybiera $X_i \dots$

A opt to S_1, S_2, S_3 :

Koszt opt: $1 + 1 + 1 = 3$

A koszt algorytmu: k - ile wynosi k ?

$$\sum_{i=1}^k |X_i| = n$$

$$\sum_{i=1}^k |X_i| + |X_k| = n$$

$$n \cdot \sum_{i=1}^{k-1} \frac{1}{2^i} = n - 4$$

$$n \left(1 - \left(\frac{1}{2}\right)^{k-1}\right) = n - 4$$

$$-n \left(\frac{1}{2}\right)^{k-1} = -4$$

$$\frac{1}{2^{k-1}} = \frac{4}{n}$$

$$n = 4 \cdot 2^{k-1}$$

$$n = 2^{k+1} \Rightarrow k = \Theta(\log n)$$

Czyli z dokładnością do stałej mamy blisko
 $\log n$ gorzej rozwiązanie.

To do:

$\rightarrow$ w notatkach KŁO jest jeszcze jeden problem z szeregowaniem,
jakoś dziwnie trudny... trzeba go tu umieścić

$\rightarrow$ dodać szeregowania zadani z ćwiczeń

$\rightarrow$ uzupełnić dowody z notatkami, szczególnie ten z Fib

$\rightarrow$ ———— Kruskala, prima i boruvka